F1TENTH / ROBORACER

RoboRacer Handbook

How to start it, drive it, test algorithms, and turn it all off.

Ben's Mac setup · September 2026 · everything runs through one command: rr

Contents

1	**Quick start**	the commands you use every time
2	**Turning things on and off**	VMs, restarting, battery, after a reboot
3	**Watching the car**	browser view, cameras, mouse controls
4	**Tracks**	practice loop, random tracks, real F1 circuits
5	**Algorithms**	run, create, tune, fix crashes
6	**Every command**	the full reference, grouped
7	**ForzaETH**	ETH Zurich's race stack
8	**The real car**	one-time setup and the safe testing routine
9	**Troubleshooting**	symptom, fix
10	**Where things live**	files and folders

TIP Forgot something? Type `rr help` in Terminal. It prints every command with a one-line explanation. Two companion PDFs sit in the same docs folder: **RoboRacer_Learning_Guide.pdf** (ROS, tuning ForzaETH, pushing to the car) and **RoboRacer_Mac_Setup_Guide.pdf** (how it was installed).

1 Quick start

Open the Terminal app and type these. That is the whole daily loop.

The daily loop

`rr sim`	Starts the simulator on Comp1 (a competition-style track) and opens it in your browser. The VM starts by itself if it was off (about 10 s).
`rr run follow_the_gap`	An algorithm takes the wheel. Watch it in the browser. Press Ctrl-C in Terminal to stop it (the car stops too).
`rr view chase`	Switch camera. Others: follow (default), top, orbit.
`rr stop`	Stops the simulator.
`rr down`	Shuts the VMs off. Do this when you are done, especially on battery.

Want a different track or your own algorithm?

`rr maps`	Lists every track.
`rr sim Comp4`	Starts on a specific track (stop the current one first with rr stop).
`rr track new`	Invents a brand-new competition-style track and tells you its name.
`rr new my_idea`	Creates your own algorithm file from a template. Edit it, then rr run my_idea.

> **TIP** Crashes are handled for you: the car is put back on the start line automatically. `rr crashes` tells you how many times it has happened.

2 Turning things on and off

The simulator runs inside a small Linux computer (a "VM") that lives on your Mac. OrbStack is the app that runs it.

Piece	What it is	When it runs
OrbStack	The app that hosts the VMs. Menu-bar icon.	Opens when any rr command needs it. Does not start at login.
VM "roboracer"	Linux + ROS 2 + the F1TENTH sim + your algorithms.	From the first rr command until rr down, quitting OrbStack, or restarting the Mac.
VM "forzaeth"	Linux + ETH Zurich's race stack.	Only when you use rr forza commands.
The sim	Physics, lidar, browser view, crash reset.	From rr sim until rr stop (or the VM stops).

Everyday on/off

`rr status`	Shows which VMs are running, whether the sim is on, and whether the car is reachable.
`rr up`	Turns the main VM on (rarely needed: every command does it automatically).
`rr down`	Turns both VMs off and frees their memory. Safe any time; nothing is lost.
`rr stop`	Stops just the simulator, leaves the VM on.
`rr forza stop`	Stops the ForzaETH sim.

Restarting (when something acts weird)

1. `rr stop` then `rr sim`: restarts just the simulator. Fixes most problems.
2. `rr down` then `rr sim`: restarts the whole VM. Fixes nearly everything else.
3. Last resort: quit OrbStack from its menu-bar icon, then run `rr sim` (it reopens OrbStack).

After you restart or shut down your Mac

Nothing to do. The VMs are off after a reboot; the next `rr sim` turns them on. Your files, tracks and algorithms are stored in your normal Documents folder, so nothing is ever lost when a VM stops.

Doing it by hand (optional)

`orbctl list`	List VMs and whether they are running.
`orbctl start roboracer`	Start one VM.
`orbctl stop roboracer forzaeth`	Stop VMs.
`open -a OrbStack`	Open the OrbStack window (you can also start and stop VMs there).

> **HEADS UP** Running VMs use memory (about 0.5 GB idle, more with the sim open) and some battery. They do not stop on their own when you close Terminal. Make rr down a habit.

3 Watching the car

The view is RViz, the standard ROS viewer, shown inside a browser tab. rr sim opens it for you.

What	Where
Your sim	http://localhost:6080 (or run rr viz)
ForzaETH sim	http://localhost:6081 (or run rr forza viz)

Camera presets

`rr view follow`	Default. Top-down, the camera follows the car. Best for watching an algorithm think.
`rr view chase`	Behind the car, like a racing game. Turns with the car.
`rr view top`	The whole track, fitted to the window. Best for watching a full lap.
`rr view orbit`	Free 3D camera you move with the mouse.

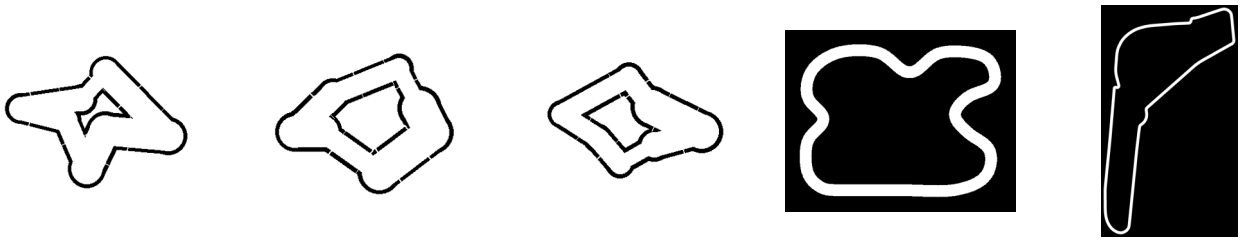
Mouse controls in the browser

Action	Does
Scroll	Zoom in and out
Drag	Rotate (orbit and chase) or pan (top and follow)
Shift + drag, or middle-drag	Pan
2D Pose Estimate button, then click-drag on the map	Teleport the car there, facing the drag direction
Views panel (right side)	Every camera setting by hand. Target Frame = what the camera follows

> **TIP** The colored dots around the car are the live lidar: every dot is where a laser beam hit a wall.

4 Tracks

Every track is a closed loop with solid walls, so the car cannot drive off the map.



Comp1 Comp4 Comp6 Squiggle Monza

Track	Size	Good for
Comp<seed> (Comp1 is default)	25 to 50 m, 1.2 to 3 m wide	Built like real RoboRacer tracks: duct walls with lidar gaps, choke points, hairpins, wide corners. ForzaETH has race lines for Comp1 to Comp8.
Squiggle	89 m lap, 1.6 m wide	Practice loop: start straight, S-bend, a tight hairpin, a chicane.
23 real F1 circuits	250 to 550 m laps	Top speed and long straights (Monza, Spa, Silverstone, ...).
levine_blocked, levine_obs	Building hallways	The classic F1TENTH lab maps.

Competition-style track generator

`rr track new`	Makes a new track with a random seed and prints its name, e.g. Comp4821.
`rr track new 42`	Same seed always gives the same track (Comp42). Share a number to race the same layout.
`rr stop; rr sim Comp42`	Drive it with your algorithms.
`rr forza addmap Comp42`	Give it to ForzaETH (computes its race line in a few seconds).

Tracks follow the official RoboRacer rules (section 2.2) and the shape of real mapped race tracks: straight runs of 33 cm air duct joined at corners, never narrower than 1.2 m (4 car widths), choke points, wide runs, extra-wide corners, sharp hairpins only where the car can make them, and small gaps in the walls the lidar sees through (like the real ducts). Every track is checked to be one clean loop with at least one duct between neighbouring sections. The start line is in the middle of the longest straight.

5 Algorithms

An algorithm is a Python file that looks at the lidar and decides speed and steering, about 40 times a second.

Commands

`rr list`	Your algorithms.
`rr run follow_the_gap`	Run one on the sim. Ctrl-C stops it and stops the car.
`rr ps`	Shows whether the sim is on and which algorithm is driving.
`rr param follow_the_gap speed_corner 0.6`	Change a setting while it drives. Takes effect instantly.
`rr new my_idea`	New algorithm from the template, built and ready to run.
`rr build`	Rebuild (only needed after adding a file by hand).

Included algorithms

Name	How it drives
follow_the_gap	Finds the closest obstacle, blocks out a bubble around it, then steers toward the deepest point of the widest open gap. Needs no map. The standard F1TENTH baseline.
wall_follow	Keeps a set distance from the left wall using a PID controller (the classic first lab).

Writing your own

`rr new my_idea` creates `ws/src/racer_algos/racer_algos/my_idea.py`. Open it in any editor on your Mac. The only part you change is `on_scan`:

```
def on_scan(self, ranges, angles, scan):
    # ranges[i] = distance (m) of laser beam i, angles[i] = its direction (rad)
    # angle 0 = straight ahead, positive = left.   self.odom = position & speed
    left  = self.range_at(ranges, angles, deg(90))
    right = self.range_at(ranges, angles, deg(-90))
    steer = self.p('gain') * (left - right) * 0.1      # steer toward the open side
    return self.p('speed'), steer                       # (speed m/s, steering rad)
```

Save, then `rr run my_idea` again. No rebuild needed after edits. Steering is automatically limited to the real car's 0.419 rad (24°), and Ctrl-C always stops the car.

Tuning follow_the_gap (and fixing "crashes at the same turn")

If the car crashes at the same corner every lap, the track is fine: the algorithm is going too fast for that corner or reacting too late. Tune it live while it drives, one setting at a time:

Setting	Default	Try	Effect
speed_corner	1.0	0.5 to 0.7	Speed when turning. Lower is the #1 fix for corner crashes.
speed_straight	2.0	1.5 to 3.0	Speed when going straight.
corner_angle_deg	10	5 to 8	How sharp a turn counts as a corner. Lower = slows down earlier.
bubble_radius	0.3	0.4 to 0.5	Safety bubble around the nearest wall. Bigger = keeps more distance.
clip_range	3.0	2.0 to 4.0	How far ahead (m) it looks. Shorter reacts to nearby turns sooner.
fov_deg	90	70 to 110	How wide (± degrees) it looks.
max_speed	2.0	any	Hard speed cap for every algorithm.

```
rr param follow_the_gap speed_corner 0.6
rr param follow_the_gap bubble_radius 0.45
```

Settings reset when the algorithm restarts. When you find good values, put them in the file (the keywords in `super().__init__(...)`) so they stick. wall_follow's settings are target_dist, lookahead, theta_deg, kp, ki and kd.

6 Every command

All commands start with rr and run in the Mac Terminal. rr help prints this list.

Simulator

`rr sim [track]`	Start sim + browser view (default Comp1).
`rr sim-opp [n]`	Sim with n cars: you plus n-1 opponents.
`rr stop`	Stop the sim.
`rr maps`	List tracks.
`rr track new [seed]`	Generate a competition-style track (Comp<seed>).
`rr viz`	Reopen the browser view.
`rr view follow|chase|top|orbit`	Camera preset.
`rr reset`	Car back to the start line now.
`rr crashes`	Crash count this session.
`rr teleop`	Drive by keyboard: i forward, comma back, j and l turn, k stop.
`rr logs`	Live sim logs (Ctrl-b d to leave, Ctrl-b n next window).

Algorithms

`rr list`	List algorithms.
`rr run <algo> [args]`	Run one on the sim.
`rr ps`	What is running.
`rr new <name>`	New algorithm from the template.
`rr param <algo> <name> <value>`	Change a setting live.
`rr build`	Rebuild the workspace.

Machine

`rr status`	VMs, sim and car status.
`rr up`	Start the main VM.
`rr down`	Stop both VMs.
`rr shell`	A Linux terminal inside the VM, ROS ready.
`rr setup`	Reinstall everything in the VM (safe to re-run).

ROS tools (advanced)

`rr topics`	List ROS topics (the data streams).
`rr echo <topic>`	Print a topic live, e.g. rr echo /drive.
`rr ros <args>`	Any ros2 command, e.g. rr ros node list.

ForzaETH

`rr forza sim [map]`	Start the ForzaETH sim (browser on port 6081).
`rr forza race [MAP|PP]`	Time trials on their optimal race line.
`rr forza h2h`	Head-to-head race with overtaking.
`rr forza laps`	Lap times so far.
`rr forza speed 0.6`	Whole-lap speed as a fraction of the race line speed (starts at 0.5).
`rr forza sector <n> <0-1>`	Speed for one sector of the lap.
`rr forza param <node> <name> <value>`	Change any setting live, e.g. controller m_l1 0.35.
`rr forza params [node]`	List a node's settings (default: controller).
`rr forza maps`	Maps ForzaETH can drive.
`rr forza addmap <track>`	Give one of your tracks to ForzaETH (a few seconds).
`rr forza viz | stop | logs | shell`	View, stop, logs, Linux shell.
`rr forza build | setup`	Rebuild or reinstall ForzaETH.

Real car

`rr car`	Log in to the car over Wi-Fi.
`rr car-key`	One-time: set up password-free login.
`rr car-bringup`	Start the car's drivers (lidar, motor, joystick).
`rr car-push`	Copy your algorithms to the car and build them.
`rr car-run <algo>`	Run an algorithm on the real car (it moves immediately).
`rr car-stop`	EMERGENCY STOP: kills the algorithm and sends zero speed.
`rr car-clean`	Clear stale DDS files (fixes std::bad_alloc at bringup).
`rr car-odom-setup`	One time: install laser odometry (this car's VESC provides none).
`rr car-odom`	Start lidar+IMU odometry. Must be running before mapping.
`rr car-check`	Checks every piece mapping needs and says which one is missing.
`rr car-map`	Start SLAM mapping (leave running while you drive a slow lap).
`rr car-map-view`	Peek at the map so far, as a picture on your Mac.
`rr car-map-save <name>`	Save the map, copy it here, make a sim + ForzaETH track from it.
`rr car-forza-setup`	Install ForzaETH on the car in Docker (one time, 10-40 min).
`rr car-forza`	Open a shell inside the car's ForzaETH container.
`rr car-topics`	List the car's ROS topics.

7 ForzaETH

The open-source race stack from ETH Zurich's team, one of the strongest in F1TENTH. It lives in its own VM.

1. `rr forza sim` (or `rr forza sim Comp1`): starts their simulator and opens http://localhost:6081. You will see their precomputed racing line in red and yellow.
2. `rr forza race`: the car drives the optimal line with their controller. `rr forza race PP` uses pure pursuit instead.
3. `rr forza h2h`: head-to-head mode with overtaking.
4. Tune while it drives: `rr forza speed 0.6`, `rr forza laps`, `rr forza param controller m_l1 0.35`. The Learning Guide explains every knob and what to change when.
5. `rr forza stop`, then `rr down` when finished.

Map	Notes
glc_ot_ez	Default. Simple loop with an overtaking zone.
GLC_smile_small	Small curvy track.
hangar_1905_v0	Their hangar test track.
Comp1 to Comp8	Your competition-style tracks, with race lines. Add more with rr forza addmap.

> **TIP** Live changes reset when the sim restarts. To keep them, edit the map's speed_scaling.yaml and config/SIM/l1_params.yaml (Learning Guide, chapter 5).

8 The real car

The car keeps its own software; your Mac just logs in over Wi-Fi. Your algorithms use the same inputs and outputs as the sim, so the same file runs on both.

Your car, as checked on 2026-09-22

Thing	Value
Address	192.168.1.145 on Wi-Fi. The 192.168.0.x address is the lidar's own wired network, not this
Login	swat-roboracer-2026, SSH key already set up, no password needed
Computer	Ubuntu 20.04, ROS 2 Foxy, 7.3 GB RAM
Workspace	~/f1tenth_ws (f1tenth_system, auto, pid_msgs)
Middleware	CycloneDDS. FastDDS kills every node with std::bad_alloc; rr sets this for you
Autonomous lock	None. Anything published on /drive moves the car at once, no button needed
ForzaETH	Needs Humble, so on this car it needs their Docker image (or their ROS 1 version)

HEADS UP This car has no deadman button: rr car-run moves it immediately. Before every run, keep a second terminal with `rr car-stop` typed and ready, and start with the wheels off the ground.

One-time setup (already done)

1. Car IP, login and workspace are in `car.env`.
2. `rr car-key` was run, so SSH needs no password.
3. Check any time with `rr status`: it should say the car is reachable. Both the Mac and the car must be on the same Wi-Fi.

Every test session

1. Battery charged. Car on a box, **wheels off the ground**, for the first run of anything new.
2. Terminal 1: `rr car-bringup`. Wait for "Streaming data" and no errors, then leave it running. If nodes die with std::bad_alloc, run `rr car-clean` and start it again.
3. Terminal 2: `rr car-push` to send and build your algorithms on the car.
4. Terminal 3: type `rr car-stop` and leave it ready to press Enter. That is your stop button.
5. Terminal 2: `rr car-run follow_the_gap -p max_speed:=0.5 -p speed_straight:=0.5 -p speed_corner:=0.4`. The car moves right away: wheels spin slowly and steer toward open space when you move a hand near the lidar.
6. Stop with Ctrl-C in terminal 2, or Enter in terminal 3.
7. Once it behaves on the box: floor, open space, nobody near, stop command ready. Then raise max_speed in steps (1.0, then 1.5).

Mapping a track (the full sequence)

Once per car: `rr car-odom-setup`. It installs lidar odometry and the sensor fusion, and touches nothing to do with the VESC.

1. Terminal 1: `rr car-bringup` (lidar, motor, joystick)
2. Terminal 2: `rr car-odom` (lidar + IMU odometry; this is what the car cannot do by itself)
3. Terminal 3: `rr car-check` every line should say OK, except /sensors/core which is the known VESC issue and does not matter
4. Terminal 3: `rr car-map` (SLAM; leave running)
5. Terminal 4: `rr car-map-view` a picture of the map so far opens on your Mac. Walls appearing = it works
6. Terminal 4: drive one slow lap: `rr car-run follow_the_gap -p max_speed:=0.6 -p speed_straight:=0.6 -p speed_corner:=0.5`, Ctrl-C at the end of the lap
7. Terminal 4: `rr car-map-save swat1` saves it, copies it to your Mac, and builds a sim track plus a ForzaETH race line from it
8. Then race your real track in the sim: `rr forza sim swat1` and `rr forza race`

> **TIP** Why the extra odometry step: SLAM must know roughly how the car moved between scans. This car's wheel data is missing (VESC firmware 6.2), so we work the motion out from the lidar and the IMU instead. If wheel data ever comes back it joins the fusion automatically.

> **HEADS UP** A real car at 2 m/s hurts when it hits a person. Start slow, never test near people or stairs, and keep the stop command within reach.

ForzaETH on the real car is a bigger project (their Docker image on the car, mapping your track, tuning a steering table). Get your own algorithms working on the car first: Learning Guide, chapter 6.

9 Troubleshooting

Symptom	Fix
Browser tab is black or says "Connecting..."	rr viz. If still stuck: rr stop, then rr sim.
Browser says the page cannot be reached	The VM is off. rr sim (or rr up, then rr viz).
"sim already running"	rr stop, then rr sim again.
Car does not move after rr run	Is the sim on? rr ps. Did the algorithm crash? Read the Terminal output.
Car crashes at the same turn every lap	Tune the algorithm (chapter 5): lower speed_corner first.
Car is stuck against a wall	rr reset, or use 2D Pose Estimate in the browser.
"unknown map"	rr maps for the exact name (capital letters matter).
Everything feels broken	rr down, then rr sim. Restarting the VM loses nothing.
Mac is slow or hot	rr down. Check that no ForzaETH race-line computation is running.
Changed an algorithm but nothing changed	Stop it (Ctrl-C) and rr run it again. New files need rr build.
rr: command not found	Open a new Terminal window. Still missing: ln -s ~/Documents/Projects/robotics/roboracer/rr /opt/homebrew/bin/rr
Car unreachable	Same Wi-Fi? Is the car on? Its Wi-Fi address is 192.168.1.145.
Car nodes die with std::bad_alloc	rr car-clean, then rr car-bringup again.
Mapping makes nothing; log says "dropping message ... frame laser"	No odometry, so SLAM cannot place the scans. Run rr car-odom first (laser odometry). The VESC (firmware 6.2) publishes no /sensors/core with the driver on the car.
No /odom or /tf on the car	Same cause. rr car-odom-setup once, then rr car-odom before mapping.

10 Where things live

Everything is in `~/Documents/Projects/robotics/roboracer`.

Path	What
`rr`	The command itself (a bash script).
`ws/src/racer_algos/racer_algos/`	**Your algorithms.** One .py file each. _base.py and _template.py are shared helpers.
`tracks/`	Every track: image, map file, centerline, start position.
`scripts/comp_track.py`	The competition-style track generator (rr track new).
`scripts/gen_track.py`	The Squiggle practice loop.
`scripts/forza_*.sh, forza_sectors.py`	ForzaETH helpers: addmap, speed, start pose, starter sectors.
`forzaeth/.../stack_master/maps/`	ForzaETH maps, each with its race line and speed_scaling.yaml.
`scripts/make_tracks.py`	Turns the downloaded F1 circuits into closed tracks.
`scripts/auto_reset.py`	Crash auto-reset.
`config/rviz/`	The camera presets.
`car.env`	The real car's address and login.
`ws/src/f1tenth_gym_ros/`	The official simulator bridge (downloaded, do not edit).
`f1tenth_gym/`	The official simulator physics (downloaded).
`forzaeth/`	ETH Zurich's race stack.
`scripts/vm_setup.sh, forza_setup.sh`	The installers (rr setup, rr forza setup).

> **TIP** Back up or share your work by copying the racer_algos folder. That is the only part that is truly yours; everything else can be reinstalled.